

XÂY DỰNG PHƯƠNG PHÁP HỎI ĐÁP TRI THỨC DỰA TRÊN BẢN THỂ HỌC (ONTOLOGY)

Võ Lê Ngọc Thịnh, Thái Phát Triển¹

¹ Viettel High Tech – trientp@viettel.com.vn

1. GIỚI THIỆU CHUNG

Trong nhiều tổ chức, tri thức nội bộ không chỉ tồn tại dưới dạng văn bản rời rạc mà còn được cấu trúc hóa thành các cơ sở tri thức (Knowledge Base) theo mô hình Bản thể học (Ontology), đặc biệt là OWL (Web Ontology Language). Cách biểu diễn này giúp máy tính có thể lưu trữ, khai thác và suy luận trên tri thức. Tuy nhiên, việc khai thác trực tiếp cơ sở tri thức dạng này gây nhiều khó khăn cho người dùng cuối, yêu cầu họ phải hiểu cấu trúc đồ thị tri thức, nắm được các thực thể, thuộc tính, lớp ontology, quan hệ giữa các thực thể và cú pháp truy vấn SPARQL.

Các mô hình ngôn ngữ lớn (LLM) mở ra hướng tiếp cận mới cho bài toán hỏi đáp bằng ngôn ngữ tự nhiên, nhưng nếu chỉ dựa vào tri thức nội tại của mô hình thì vẫn có nguy cơ sinh thông tin không chính xác. Với bài toán hỏi đáp dựa trên ontology, khó khăn không chỉ là tạo ra câu trả lời tự nhiên, mà còn là xác định đúng các thành phần cần dùng trong truy vấn. Câu hỏi của người dùng thường không trùng trực tiếp với tên gọi hoặc IRI trong knowledge graph. Nếu bước ánh xạ này sai, mô hình có thể sinh ra thuộc tính hoặc tài nguyên không tồn tại, hoặc tạo truy vấn SPARQL đúng cú pháp nhưng không khớp với schema và dữ liệu thực tế. Hình 1 minh họa vai trò của bước ánh xạ thực thể và thuộc tính khi sinh truy vấn SPARQL từ câu hỏi tự nhiên.

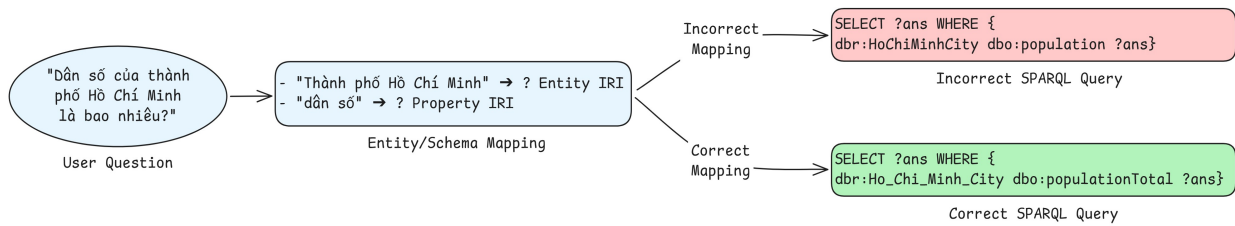
Từ thực tế trên, vấn đề đặt ra là xây dựng một hệ thống để người dùng có thể tra cứu các cơ sở tri thức phức tạp bằng ngôn ngữ tự nhiên, thay vì phải làm việc với cấu trúc

ontology và câu lệnh SPARQL. Đồng thời, quá trình trả lời cần tự động ánh xạ đúng các thành phần trong ontology như thực thể, thuộc tính và lớp ontology liên quan, truy xuất và kiểm chứng kết quả trên knowledge base nhằm nâng cao độ tin cậy của câu trả lời trong quá trình hỏi đáp.

Để giải quyết vấn đề này, đề tài xây dựng một hệ thống Hỏi đáp tri thức dựa trên Bản thể học (Ontology-based Question Answering) trên nền cơ sở tri thức DBpedia. Hệ thống được triển khai theo hướng LLM-based Agent, có khả năng phân tích câu hỏi ngôn ngữ tự nhiên, xác định các thực thể, thuộc tính và lớp ontology liên quan, sau đó sinh và thực thi truy vấn SPARQL để truy xuất câu trả lời từ knowledge base. Giải pháp này giúp giảm đáng kể rào cản kỹ thuật trong quá trình khai thác tri thức có cấu trúc, cho phép người dùng tiếp cận các kho tri thức phức tạp thông qua cách hỏi đáp tự nhiên.

Trong quá trình thực hiện đề tài, sinh viên trực tiếp tham gia nghiên cứu, thiết kế và triển khai các thành phần chính của hệ thống bao gồm:

- Xây dựng tác tử hỏi đáp dựa trên LangGraph, có khả năng phối hợp LLM với các công cụ truy xuất để từng bước sinh, kiểm tra và hiệu chỉnh truy vấn SPARQL.
- Thiết kế bộ công cụ khám phá đồ thị tri thức, gồm Search, Inspect và Execute, hỗ trợ tìm kiếm thực thể và schema, kiểm tra cấu trúc lân cận và thực thi SPARQL trên GraphDB.
- Triển khai chiến lược semantic grounding thông qua nhóm công cụ Search, nhằm ánh xạ tên riêng, thuộc



Hình 1: Minh họa yêu cầu ánh xạ thực thể và schema khi chuyển câu hỏi tự nhiên sang truy vấn SPARQL.

tính và lớp ontology trong câu hỏi tự nhiên sang các IRI tương ứng trong DBpedia, kết hợp full-text search cho thực thể và hybrid search cho schema.

- Thực hiện đánh giá hệ thống trên tập benchmark gồm 62 câu hỏi trắc nghiệm tiếng Việt. Phương pháp đề xuất đạt trung bình 51/62 câu đúng, tương ứng với độ chính xác 82.25%, cao hơn các phương pháp baseline trong thực nghiệm, bao gồm phương pháp dùng LLM để trả lời trực tiếp mà không truy xuất knowledge graph.

2. NỘI DUNG VÀ PHƯƠNG PHÁP

Dựa trên hướng tiếp cận ReAct, đề tài triển khai một hệ thống hỏi đáp cho đồ thị tri thức RDF của DBpedia theo cơ chế suy luận từng bước kết hợp với tool-calling. Cách sinh trực tiếp đáp án hoặc truy vấn SPARQL trong một lần thường thiếu cơ chế kiểm chứng từng bước với schema và dữ liệu thực tế của knowledge graph, nên dễ dẫn đến sinh truy vấn sai hoặc đưa ra câu trả lời thiếu căn cứ. Vì vậy, hệ thống cho phép mô hình lần lượt thực hiện các hành động trung gian như tìm kiếm thực thể, tìm kiếm thuộc tính hoặc lớp ontology, kiểm tra thông tin trong knowledge graph và thực thi truy vấn SPARQL.

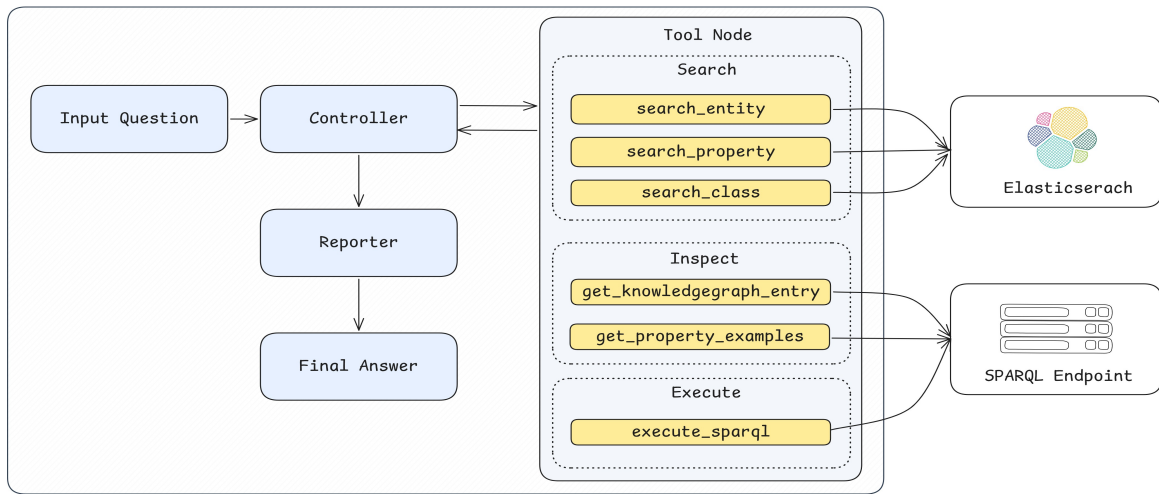
Trong quy trình trên, semantic grounding là bước quan trọng để ánh xạ các cụm từ trong câu hỏi tự nhiên sang thực thể, thuộc tính hoặc lớp tương ứng trong DBpedia. Về mặt triển khai, hệ thống sử dụng framework LangGraph để điều phối vòng lặp của agent, GraphDB làm kho RDF và SPARQL endpoint để kiểm chứng truy vấn, cùng

Elasticsearch để hỗ trợ kết hợp giữa tìm kiếm lai (hybrid search) và tìm kiếm toàn văn (full-text search). Sự kết hợp này tạo ra một vòng lặp khép kín, agent có thể liên tục đối chiếu từng giả thuyết với dữ liệu thực tế, phát hiện lỗi và điều chỉnh SPARQL cho đến khi đạt được kết quả có thể kiểm chứng.

2.1. ReAct Agent và bộ công cụ khám phá đồ thị tri thức

Trong hệ thống đề xuất, ReAct được sử dụng như cơ chế điều phối quá trình suy luận của agent. Ở mỗi vòng lặp, agent không chỉ dựa vào câu hỏi ban đầu mà còn sử dụng lịch sử các hành động đã thực hiện và các quan sát thu được từ knowledge graph để quyết định bước tiếp theo. Các hành động này được triển khai dưới dạng các công cụ mà LLM có thể gọi thông qua LangGraph. Luồng tương tác giữa controller, tool node và các hệ thống bên ngoài được minh họa trong Hình 2. Tại mỗi vòng lặp, controller có thể gọi một trong sáu công cụ truy xuất tri thức hoặc kết thúc quá trình khi đã có kết quả SPARQL đủ để trả lời câu hỏi. Cụ thể, hệ thống cung cấp ba nhóm công cụ chính:

- **Search:** Bao gồm các công cụ `search_entity`, `search_class` và `search_property`, dùng để tìm kiếm các thực thể, thuộc tính và lớp ontology liên quan đến câu hỏi. Nhóm công cụ này đóng vai trò semantic grounding, tức là ánh xạ các cụm từ trong câu hỏi sang các IRI tương ứng trong knowledge graph. Nếu không có bước này, LLM phải tự suy đoán



Hình 2: Luồng xử lý chính của Agent dựa trên LangGraph

schema của DBpedia, để sinh ra thuộc tính hoặc lớp không tồn tại, dẫn đến câu SPARQL không thực thi được hoặc không trả về kết quả. Trong đó, `search_entity` được triển khai bằng full-text search nhằm ánh xạ tên riêng sang thực thể tương ứng, còn `search_property` và `search_class` được triển khai bằng hybrid search để tìm các thuộc tính và lớp ontology phù hợp với ý nghĩa của câu hỏi.

- **Inspect:** Bao gồm các công cụ `get_knowledgegraph_entry` và `get_property_examples`, cho phép agent kiểm tra các cạnh đi ra từ một thực thể hoặc xem các ví dụ sử dụng của một thuộc tính. Nhóm công cụ này giúp agent hiểu rõ hơn cấu trúc cục bộ của knowledge graph sau khi đã tìm được các IRI ứng viên liên quan đến câu hỏi. Việc chỉ xác định được thực thể hoặc thuộc tính là chưa đủ, vì quá trình sinh SPARQL còn cần thông tin về các quan hệ lân cận, hướng liên kết và cách thuộc tính được dùng trong các triple thực tế, từ đó hạn chế việc chọn sai quan hệ hoặc sinh truy vấn không khớp dữ liệu.
- **Execute:** gồm `execute_sparql`, dùng để thực thi truy vấn SPARQL trên GraphDB và trả về kết quả thực tế từ knowledge graph.

Sau mỗi lần gọi công cụ, kết quả trả về được lưu vào trạng thái của agent. Agent tiếp tục lựa chọn hành động mới cho đến khi sinh được truy vấn SPARQL hợp lệ và có kết quả kiểm chứng. Quá trình này được giới hạn bởi một số vòng lặp tối đa nhằm tránh trường hợp agent lặp lại các hành động hoặc truy vấn không hiệu quả. Khi đã có kết quả SPARQL hợp lệ, controller dừng quá trình thăm dò và chuyển bằng chứng truy vấn sang reporter để diễn giải kết quả truy vấn.

LLM được sử dụng trong hệ thống cần có khả năng tool-calling để có thể tương tác với LangGraph và các công cụ truy xuất tri thức. Khi triển khai đề tài, mô hình Gemma 4 26B (google/gemma-4-26b-a4b-it) được lựa chọn nhờ đáp ứng tốt yêu cầu về suy luận nhiều bước, xử lý ngữ cảnh và chi phí thử nghiệm.

2.2. Chiến lược Semantic Grounding

Một vấn đề quan trọng trong các hệ thống chuyển đổi ngôn ngữ tự nhiên sang SPARQL là semantic grounding, tức là quá trình ánh xạ các cụm từ trong câu hỏi sang các IRI tương ứng trong knowledge graph. Bước này cần được thực hiện chính xác vì SPARQL không truy vấn trực tiếp trên ngôn ngữ tự nhiên, mà phụ thuộc vào các IRI cố định của thực thể, thuộc tính và lớp trong ontology. Nếu ánh xạ sai, câu truy vấn có thể vẫn đúng cú pháp nhưng dùng nhầm thực thể hoặc quan hệ, dẫn

đến không trả về kết quả hoặc trả về kết quả sai.

Trong bài toán OntologyQA, quá trình semantic grounding gồm hai nhóm chính: ánh xạ các cụm từ biểu thị thuộc tính, quan hệ hoặc lớp (như “dân số”, “nơi sinh”) sang các thành phần schema của ontology, và ánh xạ các tên riêng (như “Barack Obama”, “Hà Nội”) sang các thực thể cụ thể. Để xử lý hai nhóm này, hệ thống sử dụng hai chiến lược grounding riêng biệt trên Elasticsearch.

Đối với schema grounding: Các phân tử như `owl:Class`, `owl:ObjectProperty` và `owl:DatatypeProperty` được lập chỉ mục kèm theo nhãn, mô tả, domain và range. Do người dùng thường dùng từ đồng nghĩa hoặc diễn đạt khác với tên chính thức trong ontology, hệ thống sử dụng hybrid search, kết hợp tìm kiếm từ khóa với tìm kiếm dựa trên embedding. Kết quả từ hai hướng tìm kiếm được hợp nhất bằng thuật toán Reciprocal Rank Fusion để chọn ra các thuộc tính hoặc lớp phù hợp nhất về mặt ý nghĩa. Embedding cho mỗi phân tử schema được tạo từ văn bản tổng hợp gồm nhãn, định danh, mô tả và ngữ cảnh domain/range, sau đó đưa qua mô hình BGE-M3 để sinh vector.

Đối với named entity grounding: Hệ thống xây dựng một chỉ mục full-text search riêng trên Elasticsearch. Các thực thể được lập chỉ mục dựa trên tên gọi suy ra từ phần local name của IRI, sau khi giải mã và chuẩn hóa. Khi câu hỏi chứa tên riêng, agent gọi công cụ tìm kiếm thực thể để lấy các IRI ứng viên tương ứng. Phương pháp này phù hợp với entity grounding vì tên riêng thường phụ thuộc nhiều vào độ khớp chuỗi, trong khi số lượng thực thể trong dữ liệu lại rất lớn.

Nhờ tách riêng theo hai hướng tìm kiếm, hệ thống có thể sử dụng phương pháp phù hợp với từng loại grounding. Hybrid search được sử dụng cho schema nhằm đáp ứng yêu cầu hiểu ý nghĩa của các mối quan hệ hoặc lớp. Trong khi đó, full-text search được dành riêng cho thực thể để đảm bảo tốc độ truy xuất nhanh và khớp chính xác theo tên gọi. Ví dụ, với câu hỏi “Dân số của Thành phố

Hồ Chí Minh là bao nhiêu?”, hệ thống sử dụng full-text search để ánh xạ “Thành phố Hồ Chí Minh” sang `dbr:Ho_Chi_Minh_City`, đồng thời sử dụng hybrid search để ánh xạ “dân số” sang thuộc tính `dbo:populationTotal` trước khi tạo truy vấn SPARQL.

3. KẾT QUẢ THỰC HIỆN

Sau quá trình triển khai, đề tài đã xây dựng được một hệ thống hỏi đáp tri thức dựa trên bản thể học (Ontology-based Question Answering) có khả năng tiếp nhận câu hỏi tiếng Việt, thực hiện semantic grounding, sinh truy vấn SPARQL và kiểm chứng kết quả trên đồ thị tri thức DBpedia. Hệ thống được triển khai theo kiến trúc agentic, trong đó controller điều phối việc sử dụng các công cụ Search, Inspect và Execute để khám phá đồ thị tri thức và từng bước giải quyết vấn đề. Bảng 1 minh họa một ví dụ rút gọn về quá trình hệ thống xử lý câu hỏi. Thay vì trả lời trực tiếp, agent lần lượt tìm kiếm thực thể, xác định thuộc tính liên quan, sinh truy vấn SPARQL và kiểm chứng kết quả trên GraphDB trước khi đưa ra câu trả lời cuối cùng.

Các thành phần hệ thống đã được triển khai và tích hợp bao gồm:

- Kho RDF trên GraphDB và SPARQL Endpoint.
- Chỉ mục trên Elasticsearch phục vụ cho việc tra cứu entity (full-text search) và schema (hybrid search).
- Bộ công cụ khám phá đồ thị tri thức Search, Inspect và Execute.
- ReAct Agent được điều phối bằng LangGraph.

Hệ thống được đánh giá trên tập benchmark gồm 62 câu hỏi tiếng Việt thuộc nhiều nhóm khác nhau. Kết quả thử nghiệm được trình bày chi tiết ở phần Đánh giá hiệu quả.

Mã nguồn của hệ thống: [ngocthinh09/VDT2026-OntoAgentQA](https://github.com/ngocthinh09/VDT2026-OntoAgentQA)

Bước xử lý	Kết quả minh họa
Câu hỏi đầu vào	“Dân số của Thành phố Hồ Chí Minh là bao nhiêu?”
Tìm kiếm thực thể	<code>search_entity(“Thành phố Hồ Chí Minh”) → dbr:Ho_Chí_Minh_City</code>
Tìm kiếm thuộc tính	<code>search_property(“dân số”) → dbo:populationTotal</code>
Sinh truy vấn SPARQL	<code>SELECT ?ans WHERE { dbr:Ho_Chí_Minh_City dbo:populationTotal ?ans }</code>
Thực thi truy vấn	<code>execute_sparql(...)</code> trả về giá trị dân số từ DBpedia
Câu trả lời cuối cùng	Reporter diễn giải kết quả truy vấn thành câu trả lời cho người dùng

Bảng 1: Ví dụ minh họa quá trình xử lý câu hỏi của hệ thống

Phương pháp	Số câu đúng trung bình	Accuracy	Avg Input Tokens	Avg Output Tokens	Avg Time
LLM-Only	21.2	34.19%	175.52	6.00	0.31s
SPARQL Generation	25.2	40.65%	2560.83	357.22	6.78s
SPARQL Generation with Self-correction	33.6	54.19%	9128.15	907.16	16.93s
ReAct Agent with Semantic Grounding	51.0	82.25%	53508.71	438.63	25.22s

Bảng 2: Kết quả so sánh hiệu năng giữa phương pháp đề xuất và baselines

4. ĐÁNH GIÁ HIỆU QUẢ

Để đánh giá kết quả, đề tài sử dụng tập benchmark gồm 62 câu hỏi trắc nghiệm tiếng Việt, bao phủ nhiều dạng câu hỏi như: so sánh (comparison), suy luận đa bước (multi-hop), thực thể (entity), boolean, thuộc tính (attribute), lược đồ (schema), cực trị (superlative), danh sách (list) và đếm số lượng (counting). Hệ thống đề xuất được so sánh với 3 phương pháp baseline sử dụng cùng mô hình LLM Gemma 4 26B (google/gemma-4-26b-a4b-it).

Ba phương pháp baseline được sử dụng gồm: LLM-only, SPARQL Generation và SPARQL Generation kết hợp self-correction. Với LLM-only, mô hình nhận trực tiếp câu hỏi và các phương án trả lời, sau đó chọn đáp án dựa trên tri thức nội tại mà không truy cập knowledge graph. Với SPARQL Generation, mô hình được yêu cầu sinh câu truy vấn SPARQL từ câu hỏi, sau đó truy vấn này được thực thi trên GraphDB để lấy kết quả. Với SPARQL Generation kết hợp self-correction, hệ thống bổ sung vòng lặp phản hồi từ quá trình thực thi truy vấn. Sau mỗi lần sinh SPARQL, truy vấn được thực thi trên GraphDB và kết quả hoặc lỗi thực thi được

lưu vào lịch sử. Dựa trên lịch sử này, mô hình quyết định dừng để trả lời hoặc sinh một truy vấn SPARQL mới ở lượt tiếp theo.

Kết quả thực nghiệm ở Bảng 2 cho thấy phương pháp đề xuất đạt kết quả tốt nhất trong bốn phương pháp được so sánh, với 51/62 câu trả lời đúng, tương ứng với độ chính xác 82.25%. Trong khi đó, LLM-only đạt 34.19%, SPARQL Generation đạt 40.65% và SPARQL Generation có self-correction đạt 54.19%. Kết quả này cho thấy việc kết hợp LLM với các công cụ truy xuất knowledge graph giúp cải thiện rõ rệt khả năng trả lời so với việc chỉ dựa vào tri thức nội tại của mô hình hoặc chỉ sinh SPARQL theo một vòng lặp sửa lỗi tuyến tính.

Sự khác biệt chính nằm ở khả năng semantic grounding và khám phá dữ liệu. Với baseline SPARQL Generation, mô hình phải tự suy đoán IRI, thuộc tính và cấu trúc truy vấn nên dễ tạo ra SPARQL không khớp với schema thực tế. Self-correction giúp sửa một phần lỗi truy vấn thông qua phản hồi từ SPARQL endpoint, nhưng vẫn thiếu các công cụ để chủ động tìm thực thể, thuộc tính, lớp và kiểm tra cấu trúc lân cận trong knowledge graph. Phương pháp agentive giải quyết điểm này bằng cách cho phép mô hình gọi các

công cụ Search, Inspect và Execute trong quá trình suy luận.

Xét trên từng nhóm câu hỏi thì hệ thống đạt kết quả tốt ở các nhóm comparison, boolean, entity và attribute, do các nhóm này thường có cấu trúc truy vấn tương đối đơn giản và có thể kiểm chứng trực tiếp bằng SPARQL. Các nhóm multi-hop, schema, superlative và counting vẫn khó hơn vì yêu cầu suy luận nhiều bước, xác định đúng hướng quan hệ hoặc xử lý dữ liệu nhiều trong DBpedia.

Về chi phí thực thi, phương pháp đề xuất sử dụng trung bình 53,508.71 input tokens và 25.22 giây cho mỗi câu hỏi. Mức này cao hơn phương pháp SPARQL Generation kết hợp self-correction sử dụng 9,128.15 input tokens và 16.93 giây, cũng như cao hơn đáng kể so với LLM-only với 175.52 input tokens và 0.31 giây. Nguyên nhân là do agent phải thực hiện nhiều vòng gọi công cụ và lưu lại lịch sử quan sát trong quá trình suy luận.

Một điểm đáng chú ý là mặc dù phương pháp đề xuất đạt được độ chính xác cao hơn nhưng số output tokens trung bình sử dụng chỉ là 438.63, thấp hơn baseline SPARQL Generation kết hợp self-correction với 907.16 tokens. Điều này cho thấy chi phí chính của agent nằm ở phần ngữ cảnh đầu vào do lưu lịch sử gọi công cụ và observation, trong khi baseline self-correction tiêu tốn nhiều output tokens hơn cho quá trình sinh và sửa truy vấn SPARQL. Nhìn chung, đây là một sự đánh đổi giữa chi phí/thời gian và độ chính xác. Trong phạm vi đề tài, sự đánh đổi này là phù hợp vì mục tiêu chính là nâng cao độ chính xác trên bài toán OntologyQA.

5. KẾT LUẬN

Đề tài đã xây dựng một hệ thống hỏi đáp tri thức dựa trên bản thể học, cho phép chuyển câu hỏi ngôn ngữ tự nhiên sang quá trình truy vấn có kiểm chứng trên đồ thị tri thức. Thay vì để LLM trả lời trực tiếp, hệ thống sử dụng hướng tiếp cận ReAct kết hợp tool-calling để từng bước tìm kiếm thực thể,

thuộc tính và lớp ontology, kiểm tra cấu trúc đồ thị tri thức và thực thi truy vấn SPARQL.

Kết quả thực nghiệm trên tập benchmark 62 câu hỏi tiếng Việt cho thấy phương pháp agentic đề xuất đạt được độ chính xác 82.25%, cao hơn đáng kể các phương pháp cơ sở LLM-only, SPARQL Generation và SPARQL Generation kết hợp self-correction. Kết quả này cho thấy việc kết hợp LLM với các công cụ khám phá đồ thị tri thức giúp cải thiện đáng kể độ tin cậy của câu trả lời trong bài toán OntologyQA.

Trong tương lai, hệ thống có thể được phát triển theo một số hướng thực tiễn như: cải thiện chất lượng semantic grounding, tối ưu việc lưu trữ state và số vòng gọi công cụ để giảm chi phí và thời gian phản hồi, bổ sung cơ chế kiểm soát dữ liệu nhiều trong DBpedia, cũng như thử nghiệm trên các ontology hoặc cơ sở tri thức chuyên ngành khác. Các cải tiến này có thể giúp hệ thống tiến gần hơn tới việc ứng dụng trong các bài toán khai thác tri thức nội bộ bằng ngôn ngữ tự nhiên.

6. TÀI LIỆU THAM KHẢO

- [1] R. Cyganiak, D. Wood, and M. Lanthaler, "RDF 1.1 Concepts and Abstract Syntax," W3C Recommendation, 2014. <https://www.w3.org/TR/rdf11-concepts/>
- [2] W3C OWL Working Group, "OWL 2 Web Ontology Language Document Overview," W3C Recommendation, 2012. <https://www.w3.org/TR/owl2-overview/>
- [3] S. Harris and A. Seaborne, "SPARQL 1.1 Query Language," W3C Recommendation, 2013. <https://www.w3.org/TR/sparql11-query/>
- [4] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," arXiv:2210.03629, 2022. <https://arxiv.org/abs/2210.03629>
- [5] D. Allemang and J. F. Sequeda, "Increasing the LLM Accuracy for Question Answering: Ontologies to the Rescue!," arXiv:2405.11706, 2024. <https://arxiv.org/html/2405.11706v1>